Problem Set Solutions

Optimal Market Making for Cryptocurrency Options — The Avellaneda–Stoikov Framework and its Extension to Options

North Carolina State University · Financial Mathematics

Summer 2026

NC STATE UNIVERSITY

Problem Set Solutions

Optimal Market Making for Cryptocurrency Options

The Avellaneda–Stoikov Framework and its Extension to Options

North Carolina State University — Summer 2026

> **Confidential — for graders, and for students after each deadline.** Solutions show one correct approach with a code sketch, the expected result, and grading notes. Students may reach the same result by other means; grade on correctness, reproducibility, and code quality.

Problem Set 2 — Foundations of Market Microstructure

2.1 Order-book reconstruction (10 pts). Maintain two price→size maps. On `new`, add size; on `cancel`, subtract and **delete the level if it reaches zero**; on `modify`, replace; on `trade`, decrement the resting side that was hit.

```
from sortedcontainers import SortedDict
class L2Book:
    def __init__(self): self.bids, self.asks = SortedDict(), SortedDict()
```

```
    def _side(self, s): return self.bids if s == "bid" else self.asks
    def apply(self, e):
        bk = self._side(e["side"])
        if e["type"] in ("new", "modify"): bk[e["px"]] = e["sz"]
        elif e["type"] == "cancel":
            bk[e["px"]] = bk.get(e["px"], 0) - e["sz"]
            if bk[e["px"]] <= 0: bk.pop(e["px"], None)
        elif e["type"] == "trade":
            bk[e["px"]] = bk.get(e["px"], 0) - e["sz"]
            if bk[e["px"]] <= 0: bk.pop(e["px"], None)
    def mid(self):
        return 0.5*(self.bids.peekitem(-1)[0] + self.asks.peekitem(0)[0])
```

Expected: a clean mid/spread CSV. *Common errors:* not deleting empty levels (-2); decrementing the wrong side on a trade (-3); code does not run (≤ 3 for design).

2.2 Roll estimator (8 pts). `spread = 2·sqrt(max(0, −cov(Δp_t, Δp_{t−1})))`. It is typically **smaller** than the quoted spread because it strips adverse-selection/impact. *Errors:* sign convention on the covariance (-3); noting the discrepancy without engaging (-3).

2.3 Glosten–Milgrom quote-setter (10 pts). With informed fraction `mu` and value prior, bid/ask are posterior expectations given the trade direction. Bayesian update of the value distribution after a buy/sell; the spread is endogenous from the indifference condition. *Errors:* skipping the Bayes update (-3); bid/ask swapped (-2); not acknowledging endogeneity (-2).

2.4 GM simulation (8 pts). Average many runs; belief converges to true value. *Errors:* one realization, not averaged (-2); non-rational prior (-2).

2.5 Toxicity (9 pts). Realized spread uses a future mid; price impact is the signed mid move. *Errors:* wrong trade-direction sign (-3); discussion ignoring the actual numbers (-3).

Problem Set 3 — The Avellaneda–Stoikov Model

3.1 Reduced ODE solver (12 pts). Backward Euler on `theta'(t,q) = (alpha q^2 - ...)*theta + intensity coupling`; emit `theta` on the grid. *Errors:* wrong terminal condition (-3); algebra in the coupling (-2 ea).

3.2 Closed-form quotes (10 pts).

```
import torch
def as_quotes(S, q, gamma, sigma, kappa, tau):
```

```
r = S - q*gamma*sigma**2*tau                        # reservation price
spread = (1/gamma)*torch.log1p(gamma/kappa) + 0.5*gamma*sigma**2*tau
return r - spread, r + spread                       # bid, ask
```

Validate against 3.1 in the large-`tau` regime. *Errors:* wrong limit (−2); algebra in the half-spread (−3).

3.3 Quote engine + simulation (12 pts). Optimal policy mean-reverts inventory; symmetric baseline does not. The optimal P&L distribution has lower variance. *Errors:* baseline not actually symmetric (−3); comparing the wrong distributions (−3); does not run (≤ 4).

3.4 γ sensitivity (10 pts). Inventory variance decreases and Sharpe peaks at intermediate `gamma`. **Use a log x-axis.** *Errors:* single `gamma` (−5); linear axis (−2).

3.5 Price impact (8 pts). P&L degrades monotonically with impact. *Errors:* listing effects without quantifying (−3); no consequence for the strategy (−3).

Problem Set 4 — Stochastic Optimal Control

4.1 Merton CRRA (12 pts). Portfolio share `pi* = (mu-r)/(gammasigma^2)` constant in wealth; consumption proportional to wealth. *Errors:* wrong ansatz (−4); missing transversality (−3).

4.2 Merton CARA (10 pts). Dollar amount constant in wealth. Emit side-by-side CSV. *Errors:* failing to articulate the structural difference (−2).

4.3 Almgren–Chriss (12 pts). Optimal trajectory solves a linear ODE; hyperbolic-sinh shape; matches the discrete result. *Errors:* skipping the variational step (−4); wrong boundary conditions (−3).

4.4 HJB-residual verification (10 pts). Residual ~1e-10 for the true value function; diverges under perturbation. Use `torch.autograd.grad` for the partials. *Errors:* a "counterexample" that does not actually break the theorem (−3).

4.5 Obstacle problem (6 pts). Projected SOR; solution touches the obstacle in the contact set, matching classical theory elsewhere. *Errors:* missing the obstacle/contact set entirely (−3).

Problem Set 5 — Avellaneda–Stoikov Engine (Milestone 1)

5.1 Vectorized quote path (12 pts). Vectorize over the inventory grid; no Python loop. Report the latency distribution. *Errors:* Python grid loop (−3); reporting only the mean (−2).

5.2 Intensity calibration (12 pts). Minimize the negative log-likelihood of the exponential-intensity fill model with `torch.optim`:

```
A = torch.tensor(1.0, requires_grad=True); kap = torch.tensor(1.0, requires_grad=True)
opt = torch.optim.Adam([A, kap], lr=0.05)
for _ in range(2000):
    lam = A*torch.exp(-kap*delta)           # delta = quote distance of fills
    nll = -(torch.log(lam) - lam*dt).sum()  # Poisson log-likelihood
    opt.zero_grad(); nll.backward(); opt.step()
```

Report `A`, `kappa`, χ^2. *Errors:* OLS instead of MLE (−4); no goodness-of-fit (−3).

5.3 Paper reproduction (10 pts). Inventory oscillates around zero; P&L distribution matches the paper's shape. *Errors:* no commentary (−4).

5.4 One-month backtest (12 pts). Report all metrics vs. the symmetric baseline, with time-series plots. *Errors:* missing baseline (−4); missing a metric (−2 ea); no time series (−2).

5.5 Tests + reproducibility (4 pts). `pytest` over quote/bookkeeping; fixed-seed reproducibility. *Errors:* seed not set (−3).

Milestone 1 rubric: see Handbook Part III §3.1.

Problem Set 6 — Volatility Surface and Calibration

6.1 BS Greeks via autograd (10 pts).

```
def bs_call(S, K, T, r, sig):
    d1 = (torch.log(S/K) + (r+0.5*sig**2)*T)/(sig*torch.sqrt(T)); d2 = d1 - sig*torch.sqrt(T
    N = torch.distributions.Normal(0.,1.).cdf
    return S*N(d1) - K*torch.exp(-r*T)*N(d2)
S,K,T,r,sig = (torch.tensor(x, requires_grad=True) for x in (100.,100.,1.,0.,0.2))
price = bs_call(S,K,T,r,sig)
vega, = torch.autograd.grad(price, sig, create_graph=True)
vanna, = torch.autograd.grad(vega, S, create_graph=True)   # dvega/dS
volga, = torch.autograd.grad(vega, sig)                    # dvega/dsigma
```

Verify vanna/volga vs. finite differences. *Errors:* algebra/setup error (−2 ea); no numerical check (−3).

6.2 SVI + arbitrage checks (10 pts). Implement raw SVI total variance `w(k)=a+b(rho(k-m)+sqrt((k-m)^2+s^2))`; check butterfly (`g(k)>=0`) and calendar (monotone total variance) constraints; show which fail on a broken set. *Errors:* no constraint checks (−4); not demonstrating violations (−3).

6.3 SVI calibration (10 pts). `torch.optim` on the SVI params, autograd Jacobians; report params, per-strike residuals, RMS vol error. *Errors:* only RMS, no per-strike residuals (−3); optimizer not converged and unnoticed (−3).

6.4 SABR calibration (10 pts). Fit `alpha, beta, rho, nu` via the Hagan implied vol; compare to SVI. *Errors:* comparison not engaging strengths/weaknesses (−3); no convergence (−3).

6.5 Stability study (10 pts). Parameter time series; seeding from the prior day stabilizes the fit. *Errors:* no stability comment (−3); ineffective smoothing (−3).

Problem Set 7 — Fast Computation (C++ via `pybind11`, Milestone 2)

7.1 COS / Carr–Madan pricer (C++) (10 pts). COS: `price ~= Sigma_k Re(phi(u_k)*exp(-i u_k a))*V_k` with `u_k = kpi/(b-a)`, truncation `[a,b]` from the cumulants. Bind with `pybind11`:

```cpp
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>
namespace py = pybind11;
double cos_price(double S,double K,double T,double r,/*Heston params*/...);
PYBIND11_MODULE(fastmm, m) {
    m.def("cos_price", &cos_price, "COS European price");
}
```

Verify vs. a PyTorch Monte Carlo benchmark; report convergence vs. `N`. *Errors:* wrong truncation interval (−3); no MC verification (−3).

7.2 Robust implied vol (C++) (10 pts). Rational initial guess + safeguarded Newton/Halley; converges in the wings and at short expiry. *Errors:* unguarded Newton diverging in the wings (−4); no convergence evidence (−3).

7.3 Greeks validated by autograd (12 pts). Compare the C++ Greek vector to a PyTorch-autograd reference of the *same* pricer; max rel. error < 1e-4. **Do not hand-code adjoint AD in C++.** *Errors:* no autograd reference (−5); only first-order Greeks (−3).

7.4 Vectorized surface revaluation (C++) (10 pts). Revalue 300–500 contracts under 5 ms on one core; **release the GIL** around the C++ loop; report a latency distribution. *Errors:* GIL held during the hot loop (−3); only the mean reported (−2).

7.5 The binder, explained (8 pts). `BINDING.md` explains zero-copy tensor passing (buffer protocol / `py::array_t`), GIL release, and the CMake `find_package(pybind11)` build. *Errors:* hand-waves the GIL or the zero-copy mechanism (−3).

Milestone 2 rubric: see Handbook Part III §3.2. Latency is a hard gate.

Problem Set 8 — Options Market Making and Hedging

8.1 Multi-Greek inventory (10 pts). A fill jumps `q` by the filled option's Greek vector. Unit-test the jump. *Errors:* missing the multi-contract structure (−3); wrong aggregation (−3).

8.2 El Aoud–Abergel solver (15 pts). Reduced problem on the inventory vector with quadratic penalty `0.5 q^T Sigma q`; verify collapse to Avellaneda–Stoikov in 1-D. *Errors:* skipping the 1-D reduction check (−4); multi-dimensional algebra errors (−3 ea).

8.3 Penalty-matrix calibration (10 pts). `Sigma` from the empirical Greek covariance; justify each entry from data. *Errors:* identity matrix (−4); entries not justified (−4).

8.4 Gamma–theta–variance identity (10 pts). Hedged short-call P&L `= integral 0.5 Gamma_t (sigma^2_real - sigma^2_impl) S_t^2 dt`. Verify numerically. *Errors:* sign on theta (−2); missing the ½ factor (−2); not isolating the realized-vs-implied term (−3).

8.5 Threshold hedger + engine (10 pts). Hedge when `|Delta|>0.1 BTC`; integrate with the Week-5 engine; report cost and risk reduction. *Errors:* not integrated with the engine (−4); cost without risk reduction (−3).

Problem Set 9 — Backtesting, Risk, and Taker Flow (Milestone 3)

9.1 Event-driven backtest (12 pts). Deterministic replay; bitwise identical re-run. *Errors:* non-deterministic (−5); float drift (−2, warn).

9.2 Fill simulator (12 pts). Fill prob ≈ exponential in quote distance, parameter depends on contract liquidity; calibrate on train, validate on holdout. *Errors:* no holdout (−5); silent overfit (−3).

9.3 Full backtest (12 pts). Metrics with **block-bootstrap CI** on Sharpe; Greek time series. *Errors:* no CI (−3); Sharpe without baseline context (−2).

9.4 P&L attribution (10 pts). Streams sum to total P&L — assert it. *Errors:* attribution not summing to total (−5; always a bug).

9.5 Drawdown forensic + taker analysis (10 pts). Identify the worst-drawdown day, the driving positions, a counterfactual, and the taker flow (from the guest lecture) that hit your quotes. *Errors:* no specific positions (−4); no counterfactual (−3).

Milestone 3 rubric: see Handbook Part III §3.3.

Problem Set 10 — Advanced Extension and Final Report

10.1 Proposal (5 pts). One page, realistic scope. *Errors:* late (−3); ignores the time budget (−2).

10.2 Implementation (20 pts). Chosen extension in PyTorch on the milestone infrastructure. *Errors:* does not run (≤ 8 for design); honest partials get scaled credit.

10.3 Evaluation (15 pts). Vs. the Week-9 baseline, shared metrics. *Errors:* missing baseline (−5); cherry-picked metrics (−5).

10.4 Final report (25 pts). See the report rubric (Handbook Part III §3.4).

10.5 Final presentation (15 pts). See the presentation rubric (Handbook Part III §3.4).